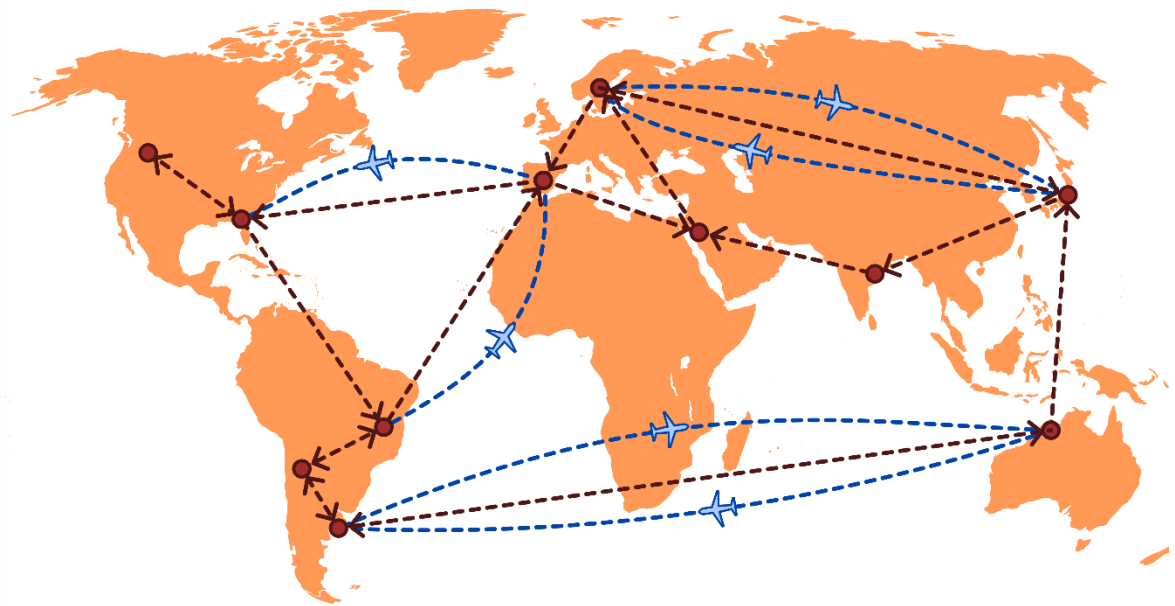


EVALUACIÓN	Obligatorio	GRUPO	TODOS	FECHA	Marzo 2026
MATERIA	Algoritmos y Estructuras de Datos				
CARRERA	Analista Programador – Analista en Tecnologías de la Información				
CONDICIONES	- Puntaje máximo: 30 puntos - Puntaje mínimo: 1 puntos - Fecha de entrega: 18/06/2026 hasta las 21:00 horas en gestion.ort.edu.uy (max. 40Mb en formato zip, rar o pdf) Inteligencia Artificial Generativa - Seguir las pautas de los docentes: Se deben seguir las instrucciones específicas de los docentes sobre cómo utilizar la IA en cada curso. - Citar correctamente las fuentes y usos de IA: Siempre que se utilice una herramienta de IA para generar contenido, se debe citar adecuadamente la fuente y la forma en que se utilizó. Se debe indicar: - o la /las herramientas utilizadas y - o el contexto de uso: por ejemplo, generación de código fuente, redacción de documentación, análisis de datos, corrección de errores, estilo, etc. - Verificar el contenido generado por la IA: No todo el contenido generado por la IA es correcto o preciso. Es esencial que los estudiantes verifiquen la información antes de usarla. - Ser responsables con el uso de la IA: Conocer los riesgos y desafíos, como la creación de “alucinaciones”, los peligros para la privacidad, las cuestiones de propiedad intelectual, los sesgos inherentes y la producción de contenido falso - El uso de herramientas de Inteligencia Artificial Generativa (IAG) puede emplearse como apoyo en el proceso de aprendizaje. Sin embargo, no sustituye el razonamiento crítico ni la elaboración personal. El estudiante es responsable de que el trabajo presentado refleje su propia comprensión, análisis y proceso cognitivo sobre el tema - En caso de existir dudas sobre la autoría, plagio o uso no atribuido de IAG, el docente tendrá la opción de convocar al equipo de obligatorio a una defensa específica e individual sobre el tema - La defensa es obligatoria y eliminatoria. - La defensa será presencial salvo en los casos en que se habilite la defensa oral sincrónica en forma remota. Defensa Fecha de defensa: Depende del grupo, a confirmar con el docente. La defensa es obligatoria y eliminatoria. El docente es quien definirá y comunicará la modalidad, y mecánica de defensa. La no presentación a la misma implica la pérdida de la totalidad de los puntos del Obligatorio. IMPORTANTE: 1) Inscribirse 2) Formar grupos de hasta 2 personas del mismo dictado 3) Subir el trabajo a Gestión antes de la hora indicada (ver hoja “RECORDATORIO” al final del documento) Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con el Coordinador o Coordinación adjunta antes de las 20:00hs. del día de la entrega, a través de los mails alamon@ort.edu.uy y rodriguez_mb@ort.edu.uy o telefónicamente al 29021505 - int 1156 u 1138				

Introducción

Se desea implementar un programa para resolver la alta demanda de las operaciones de una agencia de viaje cumpliendo con las cotas de tiempo requeridas para cada operación. Se necesita poder ingresar y consultar información sobre los pasajeros, aeropuertos, vuelos y reservas.



Todas las operaciones deberán devolver una instancia de la clase Retorno. Dicha clase contiene:

- Un resultado, que especifica si la operación se pudo realizar correctamente (OK), o si ocurrió algún error (según el número de error).
- Un valor entero, para las operaciones que retornen un número entero.
- Un valor String, para las operaciones que retornen un String, o un valor más complejo, por ejemplo, un listado, el cual será formateado según lo indicado en los ejemplos.

```
public class Retorno {  
    public enum Resultado {OK, ERROR_1, ERROR_2, ..., NO_IMPLEMENTADA};  
    public int valorEntero;  
    public String valorString;  
    public Resultado resultado;  
}
```

Se provee: una interfaz llamada Sistema, la cual no podrá ser modificada en ningún sentido, y una clase ImplementacionSistema que la implementa, donde su equipo deberá completar la implementación de las operaciones solicitadas.

Consideraciones:

- Pueden definirse tipos de datos (clases) auxiliares.
- Se provee un proyecto base, su uso es obligatorio.

Funcionalidades

01.- Inicializar Sistema

Retorno `inicializarSistema ()`;

Descripción: Inicializa las estructuras necesarias para representar el sistema especificado.

Restricción de eficiencia: no tiene.

Retornos posibles	
OK	El sistema pudo ser inicializado exitosamente.
ERROR	
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

02.- Registrar pasajero

Retorno **registrarPasajero**(String cedula, String nombre, int edad, Categoria categoria);

Descripción: Registra el pasajero con sus datos, la cédula es su identificador único.

Restricción de eficiencia: Esta operación deberá realizarse como máximo en orden $O(n)$ siendo n la cantidad total de pasajeros.

Retornos posibles	
OK	El pasajero fue registrado exitosamente.
ERROR	1. Si alguno de los parámetros es vacío o null. 2. Si la cédula no es una cédula con formato válido. 3. Si la edad es menor que cero. 4. Si ya existe un pasajero registrado con esa cédula.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Restricción: (Investigación) Se requiere el uso de expresiones regulares para lograr validar el formato de la cédula: N.NNN.NNN-N o NNN.NNN-N (No se debe realizar la validación del dígito verificador)

Las categorías posibles son:

- Platino
- Frecuente
- Estándar
- Esporádico

Ejemplos:

Resultado esperado: OK
`registrarPasajero("1.345.345-4", "Ana", 52, Categoria.PLATINO);`

Resultado esperado: ERROR_1
`registrarPasajero("1.345.345-4", null, 52, Categoria.PLATINO)`

Resultado esperado: ERROR_2
`registrarPasajero("1.3.45.345-4", "Guille", 53, Categoria.PLATINO);`

03.- Buscar Pasajero

Retorno **buscarPasajero**(String cedula);

Descripción: Retorna en valorString los datos del pasajero con el formato "cedula;nombre;edad;categoria".

Restricción de eficiencia: Esta operación deberá realizarse como máximo en orden $O(n)$, siendo n la cantidad de pasajeros en el sistema.

Retornos posibles	
OK	Si el usuario se encontró. Retorna en valorString los datos del pasajero.
ERROR	1. Si <i>cedula</i> no tiene formato válido. 2. Si no existe un pasajero registrado con esa cédula.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Formato de retorno del valorString: cedula;nombre;edad;categoria.

Por ejemplo, valorString del retorno en una consulta válida con entrada 1.345.345-4:

1.345.345-4;Pedro;25;Esporádico

04.- Listar pasajeros por cédula ascendente

Retorno **listarPasajerosAscendente**();

Descripción: Retorna en valorString los datos de todos los pasajeros registrados, ordenados por cédula (numéricamente) en forma creciente.

Restricción de eficiencia: Esta operación deberá realizarse en orden $O(n)$, siendo n la cantidad de pasajeros en el sistema.

Retornos posibles	
OK	Retornando el listado de pasajeros en valorString.
ERROR	No hay errores posibles.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Formato de retorno del valor String:

cédula1;nombre1;edad1;categoria1|cédula2;nombre2;edad2;categoria2

Nota: cédula1 es numéricamente menor a cédula2

Por ejemplo:

845.345-4;Juan;25;Esporádico|4.985.345-4;Camila;30;Frecuente

05.- Listar pasajeros por cedula descendente

Retorno **listarPasajerosDescendente()** ;

Descripción: Retorna en valorString los datos de todos los pasajeros registrados, ordenados por cédula en forma decreciente.

Restricción de eficiencia: Esta operación deberá realizarse en orden $O(n)$, siendo n la cantidad de pasajeros en el sistema.

Retornos posibles	
OK	Retornando el listado de pasajeros en valorString.
ERROR	No hay errores posibles.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Formato de retorno del valor String:

cédula1;nombre1;edad1;categoria1|cédula2;nombre2;edad2;categoria2

Por ejemplo:

4.985.345-4;Alberto;22;Frecuente|1.345.345-4;Guillermo;53;Esporádico

Nota: cédula1 es numéricamente mayor a cédula2

06.- Listar pasajeros por categoría

Retorno **listarPasajerosPorCategoría**(Categoría unaCategoría);

Descripción: Retorna en valorString los datos de todos los pasajeros registrados con esa categoría ordenados de forma creciente por cédula.

Restricción de eficiencia: Esta operación deberá realizarse en orden (k), siendo k la cantidad de pasajeros con dicha categoría.

Retornos posibles	
OK	Se pudo listar los pasajeros que pertenecen a esa categoría correctamente.
ERROR	No hay errores posibles.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Formato de retorno del valor String:

cédula1;nombre1;edad1;categoría|cédula2;nombre2;edad2;categoría

Por ejemplo:

1.345.345-;Alberto;62;Frecuente|4.985.345-4;Ana;25;Frecuente

07.- Registrar aeropuerto

Retorno **registrarAeropuerto**(String codigo, String nombre);

Descripción: Registra un aeropuerto el sistema con el código y nombre indicado. El código es el identificador único, el código y nombre no pueden ser vacíos ni null.

Restricción de eficiencia: Esta operación deberá realizarse en orden O (1).

Retornos posibles	
OK	Si el aeropuerto fue registrado exitosamente.
ERROR	1. Si alguno de los parámetros es vacío o null. 2. Si ya existe un aeropuerto con ese código.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Adicionalmente un aeropuerto tendrá vuelos esperando para despegar, los vuelos deben despegar respetando el orden de llegada.

08.- Obtener aeropuerto

Retorno **obtenerAeropuerto**(String codigo);

Descripción: Retorna en valorString los datos del aeropuerto con el formato "codigo;nombre".

Restricción de eficiencia: Esta operación deberá realizarse como máximo en orden $O(n)$, siendo n la cantidad de aeropuertos en el sistema

Retornos posibles	
OK	Si el aeropuerto se encontró. Retorna en valorString los datos del aeropuerto. Retorna en valorInt la cantidad de cantidadDeVuelosEnCola.
ERROR	1. Si código es vacío o null. 2. Si no existe un aeropuerto con ese código.
NO_IMPLEMENTADA	Cuando aún no se implementó.

09.- Registrar vuelo

Retorno **registrarVuelo**(String codigoAeropuertoOrigen, String codigoAeropuertoDestino, String codigoDeVuelo, int capacidad, int costoEnDolares);

Descripción: Registra un nuevo vuelo en el sistema con sus datos, `codigoDeVuelo` es el identificador único. Un vuelo recién registrado queda en estado "Programado", ya se pueden aceptar reservas.

Restricción de eficiencia: Esta operación no tiene restricciones de eficiencia.

Retornos posibles	
OK	Si el vuelo se registró exitosamente.
ERROR	1. Si alguno de los parámetros int es menor o igual a 0. 2. Si alguno de los parámetros String es vacío o null. 3. Si no existe el aeropuerto de origen. 4. Si no existe el aeropuerto destino. 5. Si ya existe un vuelo con ese código.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Para este obligatorio los estados de un vuelo fueron simplificados, se asume que no hay vuelos cancelados, demorados, embarcando o en curso. Adicionalmente el estado de un vuelo solo puede cambiar al siguiente.

Un vuelo puede tener los siguientes estados:

1. **Programado**: Es el estado inicial se aceptan reservas.
2. **Abierto**: Check-in habilitado y se siguen aceptando reservas.
3. **Cerrado**: No se aceptan más reservas, queda en espera para embarque y despegue.
4. **Finalizado**: Pasajeros subieron al avión y están en curso al destino.

A continuación, se muestra un diagrama de los estados de un vuelo y qué operaciones realizan el cambio de estado.



Las reservas se pueden realizar en los vuelos en estados **Programado** y **Abierto**.

El check-in solo se puede realizar en los vuelos en estado **Abierto**.

La información del vuelo se puede solicitar en todos los estados.

10.- Obtener información de un vuelo

Retorno `obtenerInformacionDeVuelo(String codigoDeVuelo);`

Descripción: Obtiene el vuelo del sistema y carga en valor string los datos con el formato:

`codigoAeropuertoOrigen;codigoAeropuertoDestino;codigoDeVuelo;capacidad;costoEnDolares;Estado;CantidadDeReservas;CantidadDePasajerosConfirmados`

Esta operación no tiene restricciones de eficiencia.

Retornos posibles	
OK	Si el vuelo se pudo obtener correctamente. Retorna en valorString los datos del vuelo con el formato indicado.
ERROR	1. Si <code>codigoDeVuelo</code> es vacío o null. 2. Si no existe un vuelo con ese código.
NO_IMPLEMENTADA	Cuando aún no se implementó.

11.- Abrir vuelo

Retorno **abrirVuelo**(String codigoDeVuelo);

Descripción: Avanza el estado de un vuelo a Abierto, un vuelo solo puede cambiar a este estado si se encuentra en estado "Programado".

Retornos posibles	
OK	Si se avanzó el estado correctamente.
ERROR	1. Si <code>codigoDeVuelo</code> es vacío o null. 2. Si no existe un vuelo con ese código. 3. Si el vuelo no está en estado "Programado".
NO_IMPLEMENTADA	Cuando aún no se implementó.

En este estado se pueden aceptar check-in de pasajeros y se siguen aceptando reservas.

12.- Cerrar vuelo

Retorno **cerrarVuelo**(String codigoDeVuelo);

Descripción: Avanza el estado de un vuelo a Cerrado, un vuelo solo puede cambiar a este estado si se encuentra en estado "Abierto". Automáticamente debe pasar a una cola de espera en el aeropuerto de origen esperando para despegue.

Se debe retornar en el valorString un listado de pasajeros confirmados (que realizaron el check-in) ordenados de forma creciente por cédula (formato idem op4). Se debe retornar en el valorInt la cantidad de pasajeros que realizaron la reserva, pero no confirmaron (realizaron check-in).

Retornos posibles	
OK	Si se avanzó el estado correctamente. Retorna en valorString los datos de los pasajeros confirmados (que realizaron el check-in ordenados creciente por cédula. Retorna en valorInt la cantidad de pasajeros que realizaron reserva y no realizaron check-in.
ERROR	1. Si <code>codigoDeVuelo</code> es vacío o null. 2. Si no existe un vuelo con ese código. 3. Si el vuelo no está en estado "Abierto".
NO_IMPLEMENTADA	Cuando aún no se implementó.

Un vuelo cerrado deja de aceptar reservas y check-in.

13.- Realizar reserva

Retorno `realizarReserva (String codigoDeVuelo, String cedula)`;

Descripción: Realiza una reserva del pasajero en el vuelo, el estado del vuelo debe ser programado o abierto y el pasajero no puede tener una reserva previa realizada para ese vuelo. La aerolínea trabaja con un 10% de overbooking por lo que se deben permitir reservas que sobrepasen la capacidad máxima (hasta ese porcentaje).

Retornos posibles	
OK	Si se realizó la reserva correctamente.
ERROR	1. Si alguno de los parámetros es vacío o null. 2. Si la cédula no tiene el formato correcto. 3. Si no existe un vuelo con ese código. 4. Si no existe un pasajero con esa cédula. 5. Si el estado del vuelo no es "Programado" o "Abierto". 6. Si el pasajero ya tiene reserva o realizó check-in en el vuelo. 7. Si se llegó a la cantidad máxima de reservas, considerando un 10% de overbooking.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Nota: El overbooking (o sobreventa) es una práctica en la que una aerolínea vende más pasajes que asientos disponibles en un vuelo. Con un 10% de overbooking, en un vuelo que tenga una capacidad de 100 lugares se debe permitir realizar 110 reservas aunque finalmente solo podrán realizar el check-in 100 pasajeros.

El límite de reservas (con overbooking) se calcula como el 110% de la capacidad y se redondea hacia arriba. Por ejemplo, con una capacidad de 78, el 10% es 7.8 por lo que se debería permitir 86 reservas.

14.- Realizar check-in

Retorno **realizarCheckIn**(String codigoDeVuelo, String cedula);

Descripción: Realiza el check-in del pasajero en el vuelo quedando el pasajero confirmado. El estado del vuelo debe ser abierto y no se puede superar la capacidad del vuelo.

Retornos posibles	
OK	Si se realizó el check-in correctamente.
ERROR	1. Si alguno de los parámetros es vacío o null. 2. Si la cédula no tiene el formato correcto. 3. Si no existe un vuelo con ese código. 4. Si no existe un pasajero con esa cédula. 5. Si el estado del vuelo no es "Abierto". 6. Si el pasajero no tiene reserva. 7. Si ya realizó el check-in. 8. Si ya se llegó a la capacidad máxima.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Para esta operación no se toma en cuenta el porcentaje de overbooking, no se debe realizar el check-in si se llegó a la capacidad máxima del vuelo.

15.- Embarque y despegue de vuelo

Retorno **embarqueYDespegueDeVuelo**(String codigoAeropuerto);

Descripción: Al cerrar un vuelo queda en espera en el aeropuerto de origen para embarcar y despegar, por lo que cada aeropuerto debe tener almacenados (por orden de llegada) los vuelos que están esperando (todos los vuelos en espera están en estado cerrado). Esta operación toma el siguiente vuelo en espera (el que está esperando hace más tiempo) y lo asigna a embarque y despegue, el estado del vuelo debe cambiar a finalizado.

Adicionalmente retorna en valorString el código de vuelo que embarcó y en valor int la cantidad restantes de vuelos.

Retornos posibles	
OK	Si el aeropuerto se encontró. Retorna en valorString el código de vuelo que embarco. Retorna en valorInt la cantidad de vuelos en cola.
ERROR	1. Si <code>codigoAeropuerto</code> es vacío o null. 2. Si no existe un aeropuerto con ese código. 3. Si no hay ningún vuelo esperando embarque y despegue en ese aeropuerto.
NO_IMPLEMENTADA	Cuando aún no se implementó.

16. Consultas disponibilidad

Retorno **consultaDisponibilidad**(int[][] matriz, int cantidad, Clase unaClase);

Descripción: Se agregó un módulo para verificar la disponibilidad de asientos en los vuelos, para esto se recibe por parámetros una matriz de int en la que están cargados los pasajeros actuales. La cantidad refiere a la cantidad de asientos contiguos (verticalmente) que se desean, la clase es un enumerado con el que se desea filtrar la clase deseada.



Retornos posibles	
OK	Si hay disponibilidad. Retorna en valorEntero la cantidad de opciones disponibles. Retorna en valorString el listado de las distintas opciones de disponibilidad para la consulta.
ERROR	1. Si la cantidad es menor o igual a 0.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Formato de retorno para una consulta para clase ejecutiva con cantidad 3:

ValorEntero: 3

ValorString: A4-B4-C4 | B4-C4-D4 | C4-D4-E4

Información importante

- **Se deberán respetar los formatos de retorno dados para las operaciones que devuelven datos. Está terminantemente prohibido el uso de clases de Java tales como ArrayList, HashMap, el uso de estas clases implica la pérdida de todos los puntos de la operación en la que se utilizaron.**
- Ninguna de las operaciones debe imprimir nada en consola.
- El sistema no debe requerir ningún tipo de interacción con el usuario por consola.
- Es obligación del estudiante mantenerse al tanto de las aclaraciones que se realicen en clase o a través del foro de aulas.
- Es necesaria una selección adecuada de las estructuras para modelar el problema y la eficiencia en cada una de las operaciones. Deberá aplicar la metodología vista en el curso, resolver el obligatorio (incluso pasando las pruebas funcionales) sin cumplir los órdenes puede implicar la pérdida del total de los puntos de la operación.
- El proyecto debe ser implementado en lenguaje JAVA en el ide NetBeans, la interfaz Sistema no se puede modificar. El uso del proyecto provisto junto a la letra es obligatorio.
- El proyecto entregado debe compilar y ejecutar correctamente en NetBeans. De no ser así implica la pérdida de todos los puntos del obligatorio. Si no se logra implementar una operación se debe dejar en un estado consistente o como "No implementada".
- No se contestarán dudas sobre el obligatorio en las 48 horas previas a la entrega.
- El equipo debe entregar un conjunto de pruebas unitarias para evidenciar una correcta estrategia de testing aplicada al proyecto.
- Dos semanas antes de la entrega la cátedra dejará disponible un conjunto de pruebas (no exhaustivas) para todas las operaciones para que puedan ajustar las desviaciones con las pruebas propias.
- Defensa: la defensa del obligatorio será en fecha y formato a confirmar por el docente. Se publicará en aulas la fecha y horario específico con antelación para cada grupo. La no asistencia a la defensa implica la pérdida de todos los puntos.
- Considere que superar las pruebas no garantiza que la operación se haya resuelto correctamente, la implementación debe respetar los órdenes solicitados.
- Entrega (Se debe subir un único zip o rar con):
 - Proyecto con las operaciones implementadas respetando las buenas prácticas y estándares vistos en el curso.
 - Incluir en la parte superior de la clase ImplementacionSistema los nombres y números de estudiante del equipo.
 - Conjunto de pruebas realizadas por el equipo.
 - Documentación: Entregar un documento PDF con una carátula con información de la materia, evaluación y del equipo. En el cuerpo del documento agregar una descripción de las estructuras utilizadas y una justificación del cumplimiento de los órdenes solicitados.
- Ninguno de los listados (que se cargan en el valorString) termina en |, ósea al agregar el último elemento no se debe agregar el |.

RECORDATORIO: IMPORTANTE PARA LA ENTREGA

• Obligatorios

La entrega de los obligatorios será en formato digital online, a excepción de algunas materias que se entregarán en Bedelía y en ese caso recibirá información específica en el dictado de la misma.

Los principales aspectos a destacar sobre la **entrega online de obligatorios** son:

1. Ingresá al sistema de Gestión.
2. En el menú, seleccioná el ítem "Evaluaciones" y la instancia de evaluación correspondiente, que figura bajo el título "Inscripto".
3. Para iniciar la entrega hacé clic en el ícono: 
4. Ingresá el número de estudiante de cada uno de los integrantes y hacé clic en "Agregar". El sistema confirmará que los integrantes estén inscriptos al obligatorio y, de ser así, mostrará el nombre y la fotografía de cada uno de ellos. Una vez agregados todos los integrantes, hacé clic en "Crear equipo".

Cualquier integrante podrá:

• Modificar la integración del equipo.

• Subir el archivo de la entrega.

5. Seleccioná el archivo que deseás entregar. Verificá el nombre del archivo que aparecerá en la pantalla y hacé clic en "Subir" para iniciar la entrega. Cada equipo (hasta 2 estudiantes) debe entregar **un único archivo en formato zip o rar** (los documentos de texto deben ser pdf, y deben ir dentro del zip o rar). El archivo a subir debe tener **un tamaño máximo de 40mb**

Cuando el archivo quede subido, se mostrará el nombre generado por el sistema (1), el tamaño y la fecha en que fue subido.

6. El sistema enviará un e-mail a todos los integrantes del equipo informando los detalles del archivo entregado y confirmando que la entrega fue realizada correctamente.
7. Podés cerrar la pestaña de entrega y continuar utilizando Gestión o salir del sistema.
8. La **hora tope para subir el archivo será las 21:00** del día fijado para la entrega.
9. La entrega se podrá realizar desde cualquier lugar (ej. hogar del estudiante, laboratorios de la Universidad, etc).
10. Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con la Coordinadora o Coordinación adjunta antes de las 20:00hs. del día de la entrega, a través de los mails, alamon@ort.edu.uy o rodriguez_mb@ort.edu.uy; o telefónicamente al 29021505 - int 1156 u 1138